

Phase 0 — Setup Runbook (Mac) — v2

Standing up the foundation for the Plank Product Data Platform. Work top to bottom — the order is deliberate. Rough total: 70–100 minutes.

What changed from v1: hosting moves from "undecided" to **GitHub Pages**, gated by **Cloudflare Access**. Refresh cadence moves from "daily to start" to **monthly**, which also changes how Supabase's idle-pause behavior needs to be handled. No Fly.io, no containers, no Dockerfile — this version has fewer moving parts than the original.

Why this order: local tools → GitHub account + repo → Supabase → Claude Project → access decision → hosting/automation. Each unblocks the next.

Your setup: macOS on Apple Silicon (M-series), zsh shell, Sublime Text, password manager as a Chrome extension. You'll create a fresh GitHub account for this work — your terminal is currently signed in to an old one, and Stage 2 fixes that so commits go to the right place. Have your Claude Team login handy.

(Note: new Terminal windows use zsh, which reads `~/ .zshrc` — that's the file the PATH lines below target. Always open a fresh window after changing it.)

Before anything else: rotate exposed Supabase credentials

If you've pasted real Supabase keys (project URL, anon key, service role key, or DB connection string) into any doc, chat, or file that isn't `infra/.env`, treat the **service role key** and **DB password** as compromised and regenerate them now:

- Supabase dashboard → **Project Settings** → **API** → regenerate the service role key
- **Project Settings** → **Database** → reset the database password

The service role key bypasses row-level security entirely, so it's the one credential worth being strict about. Going forward, real values only ever live in `infra/.env` (git-ignored) or your Chrome password manager — never in a Google Doc, even a "just us" one, since sharing permissions can widen unexpectedly.

Stage 1 — Local toolkit (~15 min)

1.1 Homebrew (the Mac package manager we'll use for everything else)

```
brew --version
```

If missing, install it (from brew.sh):

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Then add brew to your PATH. You're on Apple Silicon, so Homebrew lives at `/opt/homebrew`, and your zsh shell reads `~/ .zshrc`:

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zshrc
```

```
eval "$(/opt/homebrew/bin/brew shellenv)"
```

Confirm with `brew --version` in a new terminal window.

1.2 Git

```
git --version
```

macOS may pop up "install the command line developer tools" — accept it; that includes Git and Python. (You can set your Git identity in Stage 2, once the new account exists.)

1.3 Claude Code (native installer — no Node.js needed for Claude Code itself)

```
curl -fsSL https://claude.ai/install.sh | bash
```

Open a new terminal window (PATH updates only apply to new shells), then:

```
claude --version
```

```
claude doctor
```

Run `claude` once in any folder and log in via the browser with your Team account. If `claude --version` says "command not found", add its folder to your PATH:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc && source ~/.zshrc
```

1.4 Python 3 (for the Phase 1 sync scripts)

```
python3 --version
```

Comes with the command line tools; if missing, `brew install python`.

1.5 Node.js — no longer optional. Evidence needs it to build, and GitHub Actions will need it too (though Actions runners come with Node preinstalled — this local install is for you to build/preview on your own machine):

```
brew install node
```

1.6 (Optional) Sublime from the terminal — lets you open files/folders with `subl .`:

```
ln -s "/Applications/Sublime Text.app/Contents/SharedSupport/bin/subl" /usr/local/bin/subl
```

Stage 2 — New GitHub account + repo (~20 min)

2.1 Create a fresh GitHub account at github.com/signup — use your Plank email. Let your Chrome password-manager extension generate and save the password. Verify the email GitHub sends you.

2.2 Point your terminal at the new account (it's currently signed in as the old one)

Install the GitHub CLI:

```
brew install gh
```

Check what you're currently signed in as, then log in fresh:

```
gh auth status    # shows the OLD account, if any
```

```
gh auth login
```

In the prompts choose: GitHub.com → HTTPS → "Authenticate Git with your GitHub credentials? Yes" → Login with a web browser. Before you paste the one-time code, make sure Chrome is logged into the NEW account (open github.com in Chrome and check the top-right avatar) — the browser session is what `gh` authorises.

Set your Git identity to the new account:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "you@plankhardware.com"
```

Verify both:

```
gh auth status          # should now show the NEW account
```

```
git config --global user.email    # should show your Plank email
```

If a push later still uses the old account: the old credential is cached in the Mac Keychain. Open Keychain Access (Spotlight → "Keychain Access"), search github.com, delete the old entry, then push again — it'll re-authenticate through **gh** as the new account.

2.3 Create and clone the repo under Plank's organization (one command, now that **gh** is set up):

```
gh repo create plank-org/plank-product-platform --private --clone
```

```
cd plank-product-platform
```

(Note: this needs to land in Plank's GitHub org, not your personal account, both so Daisy/others can access it and so GitHub Pages + Actions run under org context. Confirm the org name with whoever administers it if **plank-org** isn't right.)

2.4 Scaffold it — the easy way (recommended): open Claude Code in the folder and let it build the structure.

```
claude
```

Paste this prompt:

Scaffold this repo for a data platform. Create folders /infra, /sync, /evidence, /ai,

each with a .gitkeep. Create a .gitignore that excludes .env files (but keeps

.env.example), node_modules, Python caches, /evidence/.evidence/, and a /data/ folder.

Create a README.md describing the project — a Supabase-backed product analytics

platform feeding Evidence dashboards, refreshed monthly via GitHub Actions and

published to GitHub Pages behind Cloudflare Access — and what each folder is for.

Create a `CLAUDE.md` with sections for conventions and a placeholder domain glossary.

Create an `/infra/.env.example` with placeholder keys for Supabase, Shopify (UK + US),

and Anthropic. Create a `.github/workflows` folder with a placeholder `monthly-build.yml`

(to be filled in during Stage 7). Then stage and commit everything with the message

"Phase 0: scaffold" — but do not push; I'll review first.

Review what it made (`subl .` to open it in Sublime), then push in 2.6. If you'd rather do it by hand, use the file contents in 2.5.

2.5 Scaffold it — by hand (skip if you used Claude Code)

Folders: `/infra`, `/sync`, `/evidence`, `/ai`, `/.github/workflows` (put an empty `.gitkeep` in each of the first four).

`.gitignore`:

secrets — never commit

`.env`

`.env.*`

`!.env.example`

python

`__pycache__/`

`*.pyc`

`.venv/`

`venv/`

node / evidence

`node_modules/`

`/evidence/.evidence/`

local data & builds

/data/

*.duckdb

os / editor

.DS_Store

[/infra/.env.example](#) (this file is committed — placeholders only):

Copy to /infra/.env and fill with real values. NEVER commit .env.

SUPABASE_URL=

SUPABASE_ANON_KEY=

SUPABASE_SERVICE_ROLE_KEY=

SUPABASE_DB_URL=postgresql://postgres:[PASSWORD]@[HOST]:6543/postgres

SHOPIFY_UK_STORE=

SHOPIFY_UK_ADMIN_TOKEN=

SHOPIFY_US_STORE=

SHOPIFY_US_ADMIN_TOKEN=

ANTHROPIC_API_KEY=

[CLAUDE.md](#) starter (your handover key — keep it current):

Working notes for Claude

What this repo is

Supabase (Postgres) is the central data store. Sync scripts (/sync) pull Shopify

and the Line Detail into it. Evidence (/evidence) builds the dashboards from it.

A monthly GitHub Actions workflow runs the sync, builds Evidence, and publishes to GitHub Pages, which sits behind Cloudflare Access. AI features and the merch digest live in /ai.

Conventions

- Never commit secrets; real values live in /infra/.env (git-ignored).
- SQL: snake_case tables and columns.
- Money is ex-VAT unless a column is named *_inc_vat.

Domain glossary (fill in)

- Finishes: Antique Brass, Blackened Bronze, Unlacquered Brass, Polished Nickel, Heirloom Brass
- Margin: 75% gross margin ex-VAT; charm pricing to £x.95; single blended RRP per product form
- Reconciled revenue basis: <the correct figure — state it here so it can't drift>
- Return rate = <definition>; "newness" window = <definition>; continuity = <definition>
- Collections / line names: <list>

2.6 Push and verify it's on the new account

```
git add -A
```

```
git commit -m "Phase 0: scaffold" # skip if Claude Code already committed
```

```
git push
```

Open the repo on github.com under your new account and confirm the files are there and the commit author shows your new identity. This is the check that the remote — and the right account — is the source of truth.

Stage 3 — Supabase, the data store (~20 min)

3.1 Sign up — first confirm Chrome is logged into your new GitHub account, then at supabase.com choose **Continue with GitHub** so it links the new one.

3.2 Create an organisation on the **Free plan**.

3.3 Create a new project

- Name: `plank-prod`
- Database password: generate a strong one and save it in your Chrome password manager immediately — it's shown once and can't be retrieved later.
- Region: London / eu-west-2 (or nearest EU) for UK data residency and latency.
- Plan: Free.

Wait ~2 minutes for provisioning.

3.4 Collect and store the credentials (**Project Settings** → **API** and **Database**). Save each in your Chrome password manager as a secure note — do **not** paste these into a shared doc:

- Project URL
- anon public key
- service_role secret key — treat like a root password; never in client-side code, Git, or shared docs
- Postgres connection string (from the connection/pooler panel) — used by the sync scripts

3.5 Create your local env file (holds the real secrets; git-ignored):

```
cp infra/.env.example infra/.env
```

```
subl infra/.env    # fill in the real values
```

Then confirm it's ignored — this must return nothing for `.env`:

```
git status
```

If `.env` shows up, fix `.gitignore` before committing anything.

3.6 Verify it's reachable — in the dashboard's SQL Editor, run:

```
select now();
```


A timestamp back = the store is live. (Empty is expected; schema comes in Phase 1.)

3.7 ⚠️ Idle-pause and monthly cadence — this needs a decision now, not later.

Free Supabase projects pause after 7 days of inactivity and have no backups. A **monthly** refresh cadence means the project would sit idle for ~30 days between real syncs — well past the pause threshold. Pick one:

- **Option A (recommended, still free):** add a second, lightweight GitHub Actions workflow that runs weekly and just executes `select 1;` against the database — enough to keep it awake without doing any real work. Cheap to set up, no ongoing cost.
- **Option B:** upgrade Supabase to the Pro plan, which removes auto-pause entirely. Adds a recurring cost, but also adds backups, which the Free plan lacks — worth it if this data platform becomes something you don't want to risk losing.

Whichever you pick, note it in the decisions log in Stage 6.

Stage 4 — Claude Project, the brain (~20 min)

In `claude.ai` / the Claude app (separate from Claude Code — the Project holds knowledge and planning; Claude Code does the coding).

4.1 Create a new Project — name it **Plank Product Data Platform**.

4.2 Set custom instructions — starter:

You are helping build Plank Hardware's product data platform. Use Plank's terminology and conventions (see the glossary in project knowledge). All money is ex-VAT unless stated. The reconciled revenue basis is <fill in>. Use `snake_case` in SQL. The dashboard is rebuilt monthly via GitHub Actions and published to GitHub Pages behind Cloudflare Access. When drafting specs, match the structure of the predecessor reports in project knowledge. Prefer clear, minimal solutions a less-technical colleague could maintain with your help.

4.3 Add project knowledge:

- Predecessor docs: trade report, sales reference, newness doc, returns report, and any returns-app / Yotpo exports.
- The architecture deck, the build roadmap, and this runbook.
- The domain glossary and a Line Detail data dictionary (column list). If they don't exist yet, write them now — they're the highest-value items here.

4.4 (Optional) Connect connectors — GitHub (the repo), Supabase, Google Drive (Line Detail) — if your plan offers it. Fine to skip for Phase 0.

4.5 Create one chat per workstream: Data layer / schema · Trade dashboard · Sales dashboard · Returns + reviews · Merch digest · Ops / decisions log

Stage 5 — Access-gate decision (~10 min)

Decided: Cloudflare Access, sitting in front of the GitHub Pages URL.

This isn't just the preferred option anymore — it's a required one. Plank's GitHub org is on the Free plan, which means a Pages site is always publicly reachable by its URL regardless of the repo's privacy setting. Cloudflare Access is what actually makes this private, not GitHub.

Setup:

- If Plank doesn't already have Cloudflare managing DNS for a domain you can use (e.g. a subdomain of plankhardware.com), that needs setting up first. Cloudflare's free tier covers this at your scale (up to 50 users).
- This step properly belongs with Stage 7 (once Pages is live and there's a URL to protect) — noting the decision here, executing it there.

Record this decision in the log in Stage 6.

Stage 6 — Log decisions & confirm Phase 0 is done (~5 min)

Record in the Ops / decisions log chat (or the roadmap's Decisions Log): repo URL · Supabase project name + region · Supabase idle-pause approach (keep-alive workflow vs. Pro upgrade) · access-gate choice (Cloudflare Access) · refresh cadence (**monthly**) · tracker tool (Sheets or one-seat Notion).

Definition of done — tick all eight:

- ☒ Claude Code installed and authenticated (`claude doctor clean`)
 - ☒ New GitHub account created; terminal pushes as the new account (commit author correct); repo lives under Plank's org
 - ☒ Repo created, scaffolded (README, CLAUDE.md, .gitignore, folders), pushed and visible on GitHub
 - ☒ Supabase project live; `select now();` works; credentials saved in Chrome password manager (not in shared docs); `infra/.env` created and confirmed git-ignored; idle-pause approach chosen
 - ☐ Claude Project created with predecessor docs + glossary + custom instructions; workstream chats created
 - ☐ GitHub Pages enabled (source: GitHub Actions) — see Stage 7
 - ☐ Cloudflare Access application configured and tested with a second email — see Stage 7
 - ☐ Check on Shopify and Claude API keys → small fee for Anthropic API usage to budget for
 - ☐ **Supabase idle-pause mitigation:** Chosen approach is a keep-alive workflow rather than upgrading to Supabase Pro. A monthly refresh cadence would otherwise leave the free-tier Supabase project idle past its 7-day auto-pause threshold. A separate GitHub Actions workflow (`supabase-keepalive.yml`) runs weekly, executing `select 1;` against the database to keep it active — no real work, no cost, and independent of the monthly Shopify sync/build workflow. Free-tier limitation (no backups) remains accepted at this stage; revisit if the platform moves beyond internal reporting use.
-

Stage 7 — GitHub Pages + Cloudflare Access (~20 min)

This is the piece that didn't exist in v1 — it's what turns the Evidence build into something Daisy can actually open as a link, hosted for free, with a login gate.

7.1 Enable Pages on the repo

Settings → **Pages** → **Build and deployment** → **Source: GitHub Actions**. Nothing else to configure here — this just tells GitHub to expect a deployment from the workflow below rather than a branch.

7.2 GitHub Actions workflow — replace the placeholder from Stage 2 at `.github/workflows/monthly-build.yml`:

name: Monthly build and publish

on:

schedule:

- cron: '0 6 1 * *' # 6am UTC, 1st of each month

workflow_dispatch: # lets you trigger a run manually too

permissions:

contents: read

pages: write

id-token: write

jobs:

build-and-deploy:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Sync Shopify to Supabase

run: python sync/shopify_sync.py

env:

SUPABASE_DB_URL: \${ secrets.SUPABASE_DB_URL }

SHOPIFY_UK_ADMIN_TOKEN: \${ secrets.SHOPIFY_UK_ADMIN_TOKEN }

SHOPIFY_US_ADMIN_TOKEN: \${ secrets.SHOPIFY_US_ADMIN_TOKEN }

- name: Build Evidence site

working-directory: ./evidence

run: |

npm install

npm run build

- uses: actions/upload-pages-artifact@v3

with:

path: ./evidence/evidence/build

- uses: actions/deploy-pages@v4

(`sync/shopify_sync.py` is a placeholder — rename to match whatever the Phase 1 sync script is actually called.)

7.3 Secrets to add (Settings → Secrets and variables → Actions, on the repo):

`SUPABASE_DB_URL`, `SHOPIFY_UK_ADMIN_TOKEN`, `SHOPIFY_US_ADMIN_TOKEN` — same values already sitting in `infra/.env` locally, entered here so Actions can use them without them ever appearing in code.

7.4 Weekly keep-alive workflow (resolves the Stage 3.7 idle-pause issue, if you went with Option A):

name: Weekly Supabase keep-alive

on:

schedule:

- cron: '0 6 * * 1' # every Monday, 6am UTC

jobs:

ping:

runs-on: ubuntu-latest

steps:

- name: Ping Supabase

run: |

```
psql "${{ secrets.SUPABASE_DB_URL }}" -c "select 1;"
```

7.5 Custom domain (optional but recommended) — a dashboards.plankhardware.com-style URL is easier to share and to protect than the default [.github.io](https://github.io) one:

- **Settings** → **Pages**, add the custom domain.
- In Cloudflare DNS, add a CNAME for that subdomain pointing at [<org>.github.io](https://github.io).

7.6 Cloudflare Access setup

- **Zero Trust** → **Access** → **Applications** → **Add an application** → **Self-hosted**, pointed at your Pages domain (custom subdomain, or the default [.github.io](https://github.io) URL if skipping 7.5).
- Set the policy to allow specific emails, or the whole @plankhardware.com domain.
- Test it: open the dashboard URL in a private/incognito window — you should hit an email login before seeing anything. Have a second person (or a second email you control) confirm they can get in too.

7.7 First run — trigger the workflow manually before trusting the cron:

```
gh workflow run monthly-build.yml
```

Check the Actions tab for a green run, then visit the dashboard URL to confirm the login gate and the content both work end to end.

Next: Phase 1

Design the Supabase schema and build the first Shopify + Line Detail sync — start in the **Data layer / schema** chat to draft the schema, then hand it to Claude Code to implement in [/infra](#) and [/sync](#).